

AI-assisted quality control — source code and terminal output

09_text_analysis/02_ai_quality_control.py and 04_qc_rubric_comparison.py

Landscape pages improve line length for code. Text is wrapped to **102** characters per line. Non-ASCII symbols (e.g. emoji) may appear as ? for reliable PDF fonts.

Section 1 — Full source: 02_ai_quality_control.py

```
# 02_ai_quality_control.py
# AI-Assisted Text Quality Control
# Tim Fraser

# This script demonstrates how to use AI (Ollama or OpenAI) to perform quality control
# on AI-generated text reports. It implements quality control criteria including
# boolean accuracy checks and Likert scales for multiple quality dimensions.
# Students learn to design quality control prompts and structure AI outputs as JSON.

# 0. Setup #####

## 0.1 Load Packages #####

# If you haven't already, install required packages:
# pip install pandas requests python-dotenv

import pandas as pd # for data wrangling
import re # for text processing
import requests # for HTTP requests
import json # for JSON operations
import os # for environment variables
import time
from pathlib import Path

from dotenv import load_dotenv # for loading .env file

## 0.2 Configuration #####

# Choose your AI provider: "ollama" or "openai"
AI_PROVIDER = "ollama" # Change to "openai" if using OpenAI

# Ollama configuration
PORT = 11434
OLLAMA_HOST = f"http://localhost:{PORT}"
OLLAMA_MODEL = "llama3.2:latest" # Use a model that supports JSON output

# Set True to call the API twice per report (original + revised rubric) and save
# 09_text_analysis/data/qc_prompt_ab_scores.csv for 04_qc_rubric_comparison.py
# When True, the inline demo below is skipped to avoid duplicate API calls.
EXPORT_DUAL_QC_CSV = True

# OpenAI configuration
```

```

#load_dotenv()
#OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
#OPENAI_MODEL = "gpt-4o-mini" # Low-cost model

## 0.3 Load Sample Data #####

# Load sample report text for quality control
with open("09_text_analysis/data/sample_reports.txt", "r", encoding="utf-8") as f:
    sample_text = f.read()

# Split text into individual reports
reports = [r.strip() for r in sample_text.split("\n\n") if r.strip()]
report = reports[0]

# Load source data (if available) for accuracy checking
# In this example, we'll use a simple data structure
source_data = """White County, IL | 2015 | PM10 | Time Driven | hours
|type |label_value |label_percent |
|:-----|:-----|:-----|
|Light Truck |2.7 M |51.8% |
|Car/ Bike |1.9 M |36.1% |
|Combo Truck |381.3 k |7.3% |
|Heavy Truck |220.7 k |4.2% |
|Bus |30.6 k |0.6% |"""

QC_AB_CSV = Path("09_text_analysis/data/qc_prompt_ab_scores.csv")

print("? Report for Quality Control:")
print("----")
print(report)
print("----\n")

# 1. AI Quality Control Function #####

## 1.1 Create Quality Control Prompts #####
#
# Design note (homework): we keep TWO rubrics:
# - **original** ? baseline criteria (short labels), same JSON shape as the lesson started with.
# - **revised** ? tighter instructions for factuality + one extra Likert (**completeness**).
# Three *example* improvements you could discuss in a reflection:
# (1) Tie **accurate** / **accuracy** to numbers, rankings, and unsupported claims (not just "misinter
#     preted").
# (2) Add **completeness** so the model checks coverage of the source table, not only tone.
# (3) Ask for JSON only and name keys explicitly (stricter contract, easier parsing).

def create_quality_control_prompt_original(report_text, source_data=None):
    """
    Baseline QC rubric (assignment starting point).
    """
    instructions = (
        "You are a quality control validator for AI-generated reports. "
        "Evaluate the following report text on multiple criteria and return your assessment as valid J"
        "SON."
    )

    data_context = ""

```

```

if source_data is not None:
    data_context = f"\n\nSource Data:\n{source_data}\n"

    criteria = """

    Quality Control Criteria:

    1. accurate (boolean): Verify that no part of the paragraph misinterprets the data supplied. Return TRUE if no misinterpretation. FALSE if any problems.

    2. accuracy (1-5 Likert scale): Rank the paragraph on a 5-point Likert scale, where 1 = many problems interpreting the Data vs. 5 = no misinterpretation of the Data.

    3. formality (1-5 Likert scale): Rank the paragraph on a 5-point Likert scale, where 1 = casual writing vs. 5 = government report writing.

    4. faithfulness (1-5 Likert scale): Rank the paragraph on a 5-point Likert scale, where 1 = makes grandiose claims not supported by the data vs. 5 = makes claims directly related to the data.

    5. clarity (1-5 Likert scale): Rank the paragraph on a 5-point Likert scale, where 1 = confusing writing style vs. 5 = clear and precise.

    6. succinctness (1-5 Likert scale): Rank the paragraph on a 5-point Likert scale, where 1 = unnecessarily wordy vs. 5 = succinct.

    7. relevance (1-5 Likert scale): Rank the paragraph on a 5-point Likert scale, where 1 = irrelevant commentary vs. 5 = relevant commentary about the data.

    Return your response as valid JSON in this exact format:
    {
    "accurate": true/false,
    "accuracy": 1-5,
    "formality": 1-5,
    "faithfulness": 1-5,
    "clarity": 1-5,
    "succinctness": 1-5,
    "relevance": 1-5,
    "details": "0-50 word explanation of your assessment"
    }
    """

    return f"{instructions}{data_context}\n\nReport Text to Validate:\n{report_text}{criteria}"

def create_quality_control_prompt_revised(report_text, source_data=None):
    """
    Revised rubric: sharper factuality checks + completeness Likert.
    (See module docstring / homework writeup for before-vs-after rationale.)
    """
    instructions = (
        "You are a quality control validator. Compare the report to the Source Data when provided. "
        "Return ONLY a single JSON object (no markdown fences, no extra text). "
    )

    data_context = ""
    if source_data is not None:

```

```

data_context = f"\n\nSource Data:\n{source_data}\n"

criteria = """

Quality Control Criteria (use integers 1-5 for Likert items):

1. **accurate** (boolean): TRUE only if ALL apply: (a) numeric values and percentages in the report match the Source Data (allow minor rounding); (b) relative rankings (e.g., which category is largest) match the Source Data; (c) there are no invented numbers or unsupported factual claims. Otherwise FALSE.

2. **accuracy** (1-5 Likert): 1 = multiple numerical, ranking, or exaggeration errors vs the Source Data; 3 = minor issues; 5 = fully consistent with the Source Data on numbers and ordering.

3. **formality** (1-5): 1 = casual; 5 = formal / report-like.

4. **faithfulness** (1-5): 1 = overstates differences or implies conclusions beyond the data; 5 = claims stay proportional to the evidence.

5. **clarity** (1-5): 1 = hard to follow; 5 = clear and precise.

6. **succinctness** (1-5): 1 = wordy; 5 = concise.

7. **relevance** (1-5): 1 = off-topic; 5 = focused on the data.

8. **completeness** (1-5): 1 = omits major categories or key breakdowns visible in the Source Data; 3 = mentions most; 5 = covers the main rows/categories without large gaps (if no Source Data, score based on internal consistency only).

Return JSON exactly in this shape (boolean lowercase, numbers as integers):
{
"accurate": true,
"accuracy": 3,
"formality": 3,
"faithfulness": 3,
"clarity": 3,
"succinctness": 3,
"relevance": 3,
"completeness": 3,
"details": "short explanation citing numbers or gaps if relevant"
}
"""

return f"{instructions}{data_context}\n\nReport Text to Validate:\n{report_text}{criteria}"

# Backward-compatible name used elsewhere in the course = baseline rubric
create_quality_control_prompt = create_quality_control_prompt_original

## 1.2 Query AI Function #####

# Function to query AI and get quality control results
def query_ai_quality_control(prompt, provider=AI_PROVIDER):
    if provider == "ollama":
        # Query Ollama
        url = f"{OLLAMA_HOST}/api/chat"

```

```

body = {
    "model": OLLAMA_MODEL,
    "messages": [
        {
            "role": "user",
            "content": prompt
        }
    ],
    "format": "json", # Request JSON output
    "stream": False
}

response = requests.post(url, json=body)
response.raise_for_status()
response_data = response.json()
output = response_data["message"]["content"]

elif provider == "openai":
    # Query OpenAI
    if not OPENAI_API_KEY:
        raise ValueError("OPENAI_API_KEY not found in .env file. Please set it up first.")

    url = "https://api.openai.com/v1/chat/completions"

    body = {
        "model": OPENAI_MODEL,
        "messages": [
            {
                "role": "system",
                "content": "You are a quality control validator. Always return your responses as valid JSON."
            },
            {
                "role": "user",
                "content": prompt
            }
        ],
        "response_format": {"type": "json_object"}, # Request JSON output
        "temperature": 0.3 # Lower temperature for more consistent validation
    }

    headers = {
        "Authorization": f"Bearer {OPENAI_API_KEY}",
        "Content-Type": "application/json"
    }

    response = requests.post(url, headers=headers, json=body)
    response.raise_for_status()
    response_data = response.json()
    output = response_data["choices"][0]["message"]["content"]

else:
    raise ValueError("Invalid provider. Use 'ollama' or 'openai'.")

return output

## 1.3 Parse Quality Control Results #####

```

```

def _extract_json_object(json_response):
    json_match = re.search(r"\{.*\}", json_response, re.DOTALL)
    if json_match:
        json_response = json_match.group(0)
    return json.loads(json_response)

def parse_quality_control_results(json_response, variant="original"):
    """
    Parse JSON from the model into one-row DataFrame.

    variant:
    - "original" ? 6 Likert scales + accurate + details (no completeness).
    - "revised" ? adds **completeness** Likert; overall_score uses 7 Likerts.
    """
    quality_data = _extract_json_object(json_response)

    base = {
        "accurate": [quality_data["accurate"]],
        "accuracy": [quality_data["accuracy"]],
        "formality": [quality_data["formality"]],
        "faithfulness": [quality_data["faithfulness"]],
        "clarity": [quality_data["clarity"]],
        "succinctness": [quality_data["succinctness"]],

        "relevance": [quality_data["relevance"]],
        "details": [quality_data.get("details", "")],
    }

    if variant == "original":
        base["completeness"] = [float("nan")]
        likert_cols = ["accuracy", "formality", "faithfulness", "clarity", "succinctness", "relevance"]
    else:
        base["completeness"] = [int(quality_data.get("completeness", 3))]
        likert_cols = [
            "accuracy",
            "formality",
            "faithfulness",
            "clarity",
            "succinctness",
            "relevance",
            "completeness",
        ]

    results = pd.DataFrame(base)
    shared = ["accuracy", "formality", "faithfulness", "clarity", "succinctness", "relevance"]
    results["overall_six"] = round(results[shared].astype(float).mean(axis=1), 2)
    overall = results[likert_cols].astype(float).mean(axis=1)
    results["overall_score"] = round(float(overall.iloc[0]), 2)
    return results

```

2. Run Quality Control

```

## 2.1 Demo: both rubrics on the first report (skipped when exporting CSV) #####
#####

if not EXPORT_DUAL_QC_CSV:
    print("? Querying AI ? original rubric...\n")
    quality_prompt_orig = create_quality_control_prompt_original(report, source_data)
    ai_response_orig = query_ai_quality_control(quality_prompt_orig, provider=AI_PROVIDER)
    print("? AI Response (original, raw):")
    print(ai_response_orig)
    print()

    quality_results_orig = parse_quality_control_results(ai_response_orig, variant="original")

    print("? Quality Control Results (original):")
    print(quality_results_orig)
    print()

    print("? Querying AI ? revised rubric...\n")
    quality_prompt_rev = create_quality_control_prompt_revised(report, source_data)
    ai_response_rev = query_ai_quality_control(quality_prompt_rev, provider=AI_PROVIDER)
    print("? AI Response (revised, raw):")
    print(ai_response_rev)
    print()

    quality_results_rev = parse_quality_control_results(ai_response_rev, variant="revised")
    print("? Quality Control Results (revised):")
    print(quality_results_rev)
    print()

    print(
        f"? Original overall (6 Likerts): {quality_results_orig['overall_score'].values[0]:.2f} / 5.0"
    )
    print(
        f"? Revised overall (7 Likerts incl. completeness): {quality_results_rev['overall_score'].values[0]:.2f} / 5.0"
    )
    print(f"? Fair comparison mean (shared 6 Likerts, revised): {quality_results_rev['overall_six'].values[0]:.2f}")
    print(
        f"? Accuracy check (original): {'? PASS' if quality_results_orig['accurate'].values[0] else '? FAIL'}"
    )
    print(
        f"? Accuracy check (revised): {'? PASS' if quality_results_rev['accurate'].values[0] else '? FAIL'}\n"
    )

## 2.2 Export paired scores for statistical comparison (04_qc_rubric_comparison.py) #####
#####

def export_dual_prompt_scores_to_csv(
    report_list,
    source_data_str,

```

```

csv_path: Path,
provider=AI_PROVIDER,
sleep_s: float = 1.0,
):
"""
For each report, run original and revised QC and stack rows with columns:
report_id, prompt_version, accurate, accuracy, ..., completeness, overall_score, details
"""
rows = []
for i, report_text in enumerate(report_list, start=1):
    print(f"? Export row pair {i}/{len(report_list)} (original + revised)...")
    for version, builder, variant in (
        ("original", create_quality_control_prompt_original, "original"),
        ("revised", create_quality_control_prompt_revised, "revised"),
    ):
        prompt = builder(report_text, source_data_str)
        try:
            raw = query_ai_quality_control(prompt, provider=provider)
            df = parse_quality_control_results(raw, variant=variant)
        except Exception as e:
            print(f" ? {version} report {i}: {e}")
            continue
        df.insert(0, "prompt_version", version)
        df.insert(0, "report_id", i)
        rows.append(df)
    time.sleep(sleep_s)
if not rows:
    print("?? No rows to save.")
    return
out = pd.concat(rows, ignore_index=True)
csv_path.parent.mkdir(parents=True, exist_ok=True)
out.to_csv(csv_path, index=False)
print(f"\n? Saved {len(out)} rows to {csv_path}")

if EXPORT_DUAL_QC_CSV:
    print("\n" + "=" * 60)
    print("Exporting dual-prompt QC scores for statistical comparison...")
    print("=" * 60 + "\n")
    export_dual_prompt_scores_to_csv(reports, source_data, QC_AB_CSV, provider=AI_PROVIDER)

# 3. Quality Control Multiple Reports (original rubric only) #####

## 3.1 Batch Quality Control Function #####

def check_multiple_reports(reports, source_data=None):
    print(f"? Performing quality control on {len(reports)} reports...\n")

    all_results = []

    for i, report_text in enumerate(reports, 1):
        print(f"Checking report {i} of {len(reports)}...")

    # Create prompt
    prompt = create_quality_control_prompt_original(report_text, source_data)

```



```

# Query AI
try:
    response = query_ai_quality_control(prompt, provider=AI_PROVIDER)
    results = parse_quality_control_results(response, variant="original")
    results["report_id"] = i
    all_results.append(results)
except Exception as e:
    print(f"? Error checking report {i}: {e}")

time.sleep(1)

# Combine all results
if all_results:
    combined_results = pd.concat(all_results, ignore_index=True)
    return combined_results
else:
    return pd.DataFrame()

## 3.2 Run Batch Quality Control (Optional) #####

# Uncomment to check all reports
# if len(reports) > 1:
#     batch_results = check_multiple_reports(reports, source_data)
#     print("\n? Batch Quality Control Results:")
#     print(batch_results)

print("? AI quality control complete!")
print("? Compare these results with manual quality control (01_manual_quality_control.py) to see how A
I performs.")
print(f"? For Bartlett / paired t-tests on original vs revised rubric, run 04_qc_rubric_comparison.py
(reads {QC_AB_CSV}).")

```

Section 2 — Terminal output: *02_ai_quality_control.py* (demo mode)

How this section was produced. The demo runs when **EXPORT_DUAL_QC_CSV = False** (both rubrics on the first report).

Source: data/pdf_embedded_02_demo_stdout.txt (bundled snapshot). For a fresh capture, run this script without --skip-ollama with Ollama up, or save stdout to data/captured_02_stdout.txt.

```
? Report for Quality Control:
---
In 2015, Light Trucks contributed 51.8% of PM10 emissions in White County, IL. Cars/Bikes accounted for 36.1% of total PM10 emissions in the area that year. Combo Trucks, Heavy Trucks, and Buses combined were responsible for only 12.1% of PM10 emissions. To lower PM10 emissions in White County, IL, focus should be on reducing emissions from Light Trucks and Cars/Bikes, as they collectively contribute to 87.9% of total emissions. Implementing stricter emission standards for these vehicles or promoting the use of electric vehicles could help significantly reduce pollution levels in the region.
---

? Querying AI ? original rubric...

? AI Response (original, raw):
{
  "accurate": true,
  "accuracy": 4,
  "formality": 5,
  "faithfulness": 5,
  "clarity": 5,
  "succinctness": 4,
  "relevance": 5,
  "details": "The report accurately presents the data from White County, IL for PM10 emissions in 2015. The text correctly identifies Light Trucks and Cars/Bikes as contributing to most of the emissions, and suggests focusing on reducing emissions from these sources to lower pollution levels. However, it could be slightly more concise without losing any important information."
}

? Quality Control Results (original):
accurate accuracy formality ... completeness overall_six overall_score
0 True 4 5 ... NaN 4.67 4.67

[1 rows x 11 columns]

? Querying AI ? revised rubric...

? AI Response (revised, raw):
{
  "accurate": true,
  "accuracy": 3,
  "formality": 5,
  "faithfulness": 3,
  "clarity": 3,

  "succinctness": 4,
  "relevance": 5,
  "completeness": 3,
  "details": "Report numbers match the Source Data (with minor rounding), and relative rankings are consistent."
}
```

```
? Quality Control Results (revised):
accurate accuracy formality ... completeness overall_six overall_score
0 True 3 5 ... 3 3.83 3.71
```

```
[1 rows x 11 columns]
```

```
? Original overall (6 Likerts): 4.67 / 5.0
? Revised overall (7 Likerts incl. completeness): 3.71 / 5.0
? Fair comparison mean (shared 6 Likerts, revised): 3.83
? Accuracy check (original): ? PASS
? Accuracy check (revised): ? PASS
```

```
? AI quality control complete!
? Compare these results with manual quality control (01_manual_quality_control.py) to see how AI perfo
rms.
? For Bartlett / paired t-tests on original vs revised rubric, run 04_qc_rubric_comparison.py (reads 0
9_text_analysis/data/qc_prompt_ab_scores.csv).
```

Section 3 — Full source: *04_qc_rubric_comparison.py*

```
# 04_qc_rubric_comparison.py
# Compare original vs revised QC rubric scores (paired reports)
# Tim Fraser / course extension
#
# Input: 09_text_analysis/data/qc_prompt_ab_scores.csv
# (generate with 02_ai_quality_control.py and EXPORT_DUAL_QC_CSV = True)
#
# Run from repo root:
# python3 09_text_analysis/04_qc_rubric_comparison.py

from pathlib import Path
import sys
import warnings

import numpy as np
import pandas as pd
from scipy.stats import bartlett
import pingouin as pg

QC_AB_PATH = Path("09_text_analysis/data/qc_prompt_ab_scores.csv")

# Human-readable labels (what ?original? and ?revised? mean in this pipeline)
LABEL_ORIGINAL = "Original QC rubric (baseline lesson prompt)"
LABEL_REVISIED = "Revised QC rubric (stricter factuality + completeness)"

def _pingouin_col(df, *names):
    """Pingouin renamed hyphenated column names to underscores in newer versions."""
    for n in names:
        if n in df.columns:
            return df[n].values[0]
    raise KeyError(f"Expected one of {names}; columns={list(df.columns)}")

def _conclusion_text(mean_diff: float, p_t: float, dim: str) -> str:
    """Plain-language takeaway for paired comparison (Revised minus Original)."""
    if np.isnan(p_t):
        return (
            "We cannot run a paired t-test here (e.g. every report got the same score under both rubri"
            "cs). "
            "There is no variation in paired differences to test."
        )
    if p_t >= 0.05:
        return (
            f"No statistically significant shift between rubrics on **{dim}** (paired p = {p_t:.4f})."
            "Any mean gap you see could plausibly be random noise at usual ? = 0.05."
        )
    if mean_diff > 0:
        return (
            "The **revised** scoring rubric gave **significantly higher** mean scores than the **origi"
            "nal** rubric "
            f"on **{dim}** (paired p = {p_t:.4f}). On this scale, a higher number is a stronger/better"
        )
```

```

rating."
)
if mean_diff < 0:
    return (
        "The **original** scoring rubric gave **significantly higher** mean scores than the **revised** rubric "
        f"on **{dim}** (paired p = {p_t:.4f}). "
        "So the revised rubric produced lower average scores on this criterion for these reports."
    )
    return f"Paired p = {p_t:.4f}, but the mean difference is effectively zero."

def main() -> None:
    if not QC_AB_PATH.is_file():
        print(f"? File not found: {QC_AB_PATH}")
        print(
            " Generate it: set EXPORT_DUAL_QC_CSV = True in 02_ai_quality_control.py, "
            "then run that script from the repo root."
        )
        sys.exit(1)

    ab = pd.read_csv(QC_AB_PATH)

    print("=" * 72)
    print(" QC RUBRIC COMPARISON ? what this script does")
    print("=" * 72)
    print(
        "Each row in your CSV is one report scored with ONE scoring prompt.\n"
        f"? {LABEL_ORIGINAL}\n"
        f"? {LABEL_REVISSED}\n"
        "\n"
    )

    "The **same** report text is evaluated twice (paired). We report, for each numeric criterion:\n"
    " 1. Scores under each rubric (per report, then mean & standard deviation)\n"
    " 2. Bartlett's test ? checks whether spread (variance) differs between the two rubrics? scores\n"
    " 3. Paired t-test ? tests whether **mean** scores differ between the two rubrics on the same reports\n"
    " 4. A short conclusion in everyday language\n"
    )
    print(f"Data file: {QC_AB_PATH.resolve()}")
    print(f"Total rows in file: {len(ab)}")
    print("=" * 72 + "\n")

    if ab["accurate"].dtype == object:
        ab["accurate"] = ab["accurate"].map(lambda x: str(x).lower() in ("true", "1", "yes"))

    orig = ab[ab["prompt_version"] == "original"].sort_values("report_id").reset_index(drop=True)
    rev = ab[ab["prompt_version"] == "revised"].sort_values("report_id").reset_index(drop=True)

    if len(orig) == 0 or len(rev) == 0:
        print("??? Need both prompt_version == 'original' and 'revised' rows.")
        sys.exit(1)

    if len(orig) != len(rev) or not orig["report_id"].equals(rev["report_id"]):
        print("??? original/revised rows do not align by report_id; check the CSV.")

```

```

sys.exit(1)

n = len(orig)
print(f"Number of paired reports: {n}\n")

# Overview table
print("-" * 72)
print("OVERVIEW ? mean of each numeric column by rubric")
print("-" * 72)
num_cols = [
c
for c in ab.columns
if c not in ("report_id", "prompt_version", "details")
and pd.api.types.is_numeric_dtype(ab[c])
]
desc = ab.groupby("prompt_version")[num_cols].agg(["mean", "std"]).round(3)
print(desc)

print()

paired_dims = [
"accuracy",
"formality",
"faithfulness",
"clarity",
"succinctness",
"relevance",
"overall_six",
"overall_score",
]

print(
"Note: **overall_score** = mean of 6 Likerts (original) vs 7 (revised, includes completeness).
"
"***overall_six** = mean of the **same six** Likerts for both ? best for apples-to-apples.\n"
)
print(
"Caution: with very few reports, Likert scores often repeat ? variance can be zero ? "
"Bartlett may be unreliable or nan; conclusions lean on means and common sense.\n"
)

for dim in paired_dims:
if dim not in orig.columns:
continue

x = orig[dim].astype(float).values
y = rev[dim].astype(float).values
mean_diff = float(np.mean(y - x))
diffs = y - x

mo, so = float(np.mean(x)), float(np.std(x, ddof=1)) if n > 1 else 0.0
mr, sr = float(np.mean(y)), float(np.std(y, ddof=1)) if n > 1 else 0.0

print("=" * 72)
print(f" MEASURE: {dim}")
print("=" * 72)

```

```

print("\nPer-report scores (same report_id = same text, two different scoring prompts):\n")
print(f"{'report_id':>10} {'Original':>12} {'Revised':>12} {'Revised ? Original':>18}")
for i in range(n):

    rid = int(orig["report_id"].iloc[i])
    print(f"{rid:>10} {x[i]:>12.3f} {y[i]:>12.3f} {diffs[i]:>+18.3f}")

print("\n--- Summary statistics ---\n")
print(f" {LABEL_ORIGINAL}")
print(f" Mean = {mo:.4f} Std dev = {so:.4f} (n = {n} reports)\n")
print(f" {LABEL_REVISSED}")
print(f" Mean = {mr:.4f} Std dev = {sr:.4f} (n = {n} reports)\n")
print(f" Mean of (Revised ? Original) across reports: {mean_diff:+.4f}")

# Bartlett
with warnings.catch_warnings(record=True):
    warnings.simplefilter("always")
    try:
        b_stat, b_p = bartlett(x, y)
        b_p = float(b_p)
    except Exception:
        b_stat, b_p = float("nan"), float("nan")

print("\n--- Bartlett test (are the two rubrics? score spreads different?) ---\n")
if not np.isnan(b_p) and not np.isinf(b_stat):
    print(f" Test statistic: {b_stat:.6f}")
    print(f" p-value: {b_p:.6f}")
    if b_p < 0.05:
        print(
            " Meaning: variances differ significantly across rubrics (? = 0.05). "
            "Interpret other tests with that in mind."
        )
    else:
        print(
            " Meaning: we do **not** reject equal spread at ? = 0.05 (often a loose check with small n)."
        )
    else:
        print(
            " Bartlett is not reliable here (non-finite statistic or nan p-value ? typical when one rubric has zero spread, e.g. all the same Likert value). Do not use this p-value to draw conclusions."
        )

# Paired t

print("\n--- Paired t-test (do mean scores differ between rubrics on the same reports?) ---\n")
p_t = float("nan")
t_val = float("nan")
dof = float("nan")
if np.allclose(diffs, 0.0):
    print(" Not run: Revised and Original scores are **identical** for every report on this measure.")
    print(" (No variation in paired differences ? t-test is not defined.)\n")
else:

```

```

try:
    tt = pg.ttest(x, y, paired=True)
    p_t = float(_pingouin_col(tt, "p-val", "p_val"))
    t_val = float(tt["T"].values[0])
    dof = float(tt["dof"].values[0])
    print(f" t statistic: {t_val:.6f}")
    print(f" degrees of freedom: {dof:.1f}")
    print(f" two-sided p-value: {p_t:.6f}\n")
    print(" Full Pingouin table:")
    print(tt.to_string(index=False))
    print()
except Exception as exc:
    print(f" Could not compute paired t-test: {exc}\n")

print("--- What to say in plain language ---\n")
print(f" {_conclusion_text(mean_diff, p_t, dim)}\n")

if "completeness" in rev.columns and rev["completeness"].notna().any():
    print("=" * 72)
    print(" EXTRA: completeness (only defined for the revised rubric)")
    print("=" * 72)
    print(
        f" Mean completeness score under the revised rubric: {rev['completeness'].mean():.4f}\n"
        " (There is no separate ?completeness? column for the original rubric.)\n"
    )

print("=" * 72)
print(" Done.")
print("=" * 72)

if __name__ == "__main__":
    main()

```


Section 4 — Terminal output: *04_qc_rubric_comparison.py*

Requires *09_text_analysis/data/qc_prompt_ab_scores.csv* (from 02 with **EXPORT_DUAL_QC_CSV = True**).

```
=====
QC RUBRIC COMPARISON ? what this script does
=====
Each row in your CSV is one report scored with ONE scoring prompt.
? Original QC rubric (baseline lesson prompt)
? Revised QC rubric (stricter factuality + completeness)

The **same** report text is evaluated twice (paired). We report, for each numeric criterion:
1. Scores under each rubric (per report, then mean & standard deviation)
2. Bartlett's test ? checks whether spread (variance) differs between the two rubrics? scores
3. Paired t-test ? tests whether **mean** scores differ between the two rubrics on the same reports
4. A short conclusion in everyday language

Data file: /Users/mjt/Desktop/SYSEN/AIDATA/dsai/09_text_analysis/data/qc_prompt_ab_scores.csv
Total rows in file: 8
=====

Number of paired reports: 4

-----
OVERVIEW ? mean of each numeric column by rubric
-----
accurate accuracy ... overall_six overall_score
mean std mean ... std mean std
prompt_version ...
original 1.0 0.0 4.25 ... 0.704 4.165 0.704
revised 1.0 0.0 3.00 ... 0.290 4.140 0.351

[2 rows x 20 columns]

Note: **overall_score** = mean of 6 Likerts (original) vs 7 (revised, includes completeness). **overall_six** = mean of the **same six** Likerts for both ? best for apples-to-apples.

Caution: with very few reports, Likert scores often repeat ? variance can be zero ? Bartlett may be unreliable or nan; conclusions lean on means and common sense.

=====
MEASURE: accuracy
=====

Per-report scores (same report_id = same text, two different scoring prompts):

report_id Original Revised Revised ? Original
1 5.000 3.000 -2.000
2 4.000 3.000 -1.000
3 4.000 3.000 -1.000
4 4.000 3.000 -1.000

--- Summary statistics ---

Original QC rubric (baseline lesson prompt)
Mean = 4.2500 Std dev = 0.5000 (n = 4 reports)
```

Revised QC rubric (stricter factuality + completeness)
Mean = 3.0000 Std dev = 0.0000 (n = 4 reports)

Mean of (Revised ? Original) across reports: -1.2500

--- Bartlett test (are the two rubrics? score spreads different?) ---

Bartlett is not reliable here (non-finite statistic or nan p-value ? typical when one rubric?s score s have zero spread, e.g. all the same Likert value). Do not use this p-value to draw conclusions.

--- Paired t-test (do mean scores differ between rubrics on the same reports?) ---

t statistic: 5.000000
degrees of freedom: 3.0
two-sided p-value: 0.015392

Full Pingouin table:

T	dof	alternative	p_val	CI95	cohen_d	power	BF10
5.0	3	two-sided	0.015392	[0.45, 2.05]	3.535534	0.992404	5.272

--- What to say in plain language ---

The **original** scoring rubric gave **significantly higher** mean scores than the **revised** rubric on **accuracy** (paired p = 0.0154). So the revised rubric produced lower average scores on this criterion for these reports.

=====

MEASURE: formality

=====

Per-report scores (same report_id = same text, two different scoring prompts):

report_id	Original	Revised	Revised ? Original
1	4.000	5.000	+1.000
2	4.000	4.000	+0.000
3	3.000	5.000	+2.000
4	2.000	4.000	+2.000

--- Summary statistics ---

Original QC rubric (baseline lesson prompt)
Mean = 3.2500 Std dev = 0.9574 (n = 4 reports)

Revised QC rubric (stricter factuality + completeness)
Mean = 4.5000 Std dev = 0.5774 (n = 4 reports)

Mean of (Revised ? Original) across reports: +1.2500

--- Bartlett test (are the two rubrics? score spreads different?) ---

Test statistic: 0.631585
p-value: 0.426775
Meaning: we do **not** reject equal spread at ? = 0.05 (often a loose check with small n).

--- Paired t-test (do mean scores differ between rubrics on the same reports?) ---

```
t statistic: -2.611165
degrees of freedom: 3.0
two-sided p-value: 0.079605
```

Full Pingouin table:

```
T dof alternative p_val CI95 cohen_d power BF10
-2.611165 3 two-sided 0.079605 [-2.77, 0.27] 1.581139 0.572567 1.736
```

--- What to say in plain language ---

No statistically significant shift between rubrics on **formality** (paired p = 0.0796). Any mean gap you see could plausibly be random noise at usual $\alpha = 0.05$.

```
=====
MEASURE: faithfulness
=====
```

Per-report scores (same report_id = same text, two different scoring prompts):

```
report_id Original Revised Revised ? Original
1 5.000 4.000 -1.000
2 5.000 3.000 -2.000
3 5.000 5.000 +0.000
4 3.000 2.000 -1.000
```

--- Summary statistics ---

Original QC rubric (baseline lesson prompt)
Mean = 4.5000 Std dev = 1.0000 (n = 4 reports)

Revised QC rubric (stricter factuality + completeness)
Mean = 3.5000 Std dev = 1.2910 (n = 4 reports)

Mean of (Revised ? Original) across reports: -1.0000

--- Bartlett test (are the two rubrics' score spreads different?) ---

```
Test statistic: 0.165956
p-value: 0.683731
Meaning: we do not reject equal spread at  $\alpha = 0.05$  (often a loose check with small n).
```

--- Paired t-test (do mean scores differ between rubrics on the same reports?) ---

```
t statistic: 2.449490
degrees of freedom: 3.0
two-sided p-value: 0.091721
```

Full Pingouin table:

```
T dof alternative p_val CI95 cohen_d power BF10
2.44949 3 two-sided 0.091721 [-0.3, 2.3] 0.866025 0.231478 1.577
```

--- What to say in plain language ---

No statistically significant shift between rubrics on **faithfulness** (paired p = 0.0917). Any mean gap you see could plausibly be random noise at usual $\alpha = 0.05$.

```
=====
MEASURE: clarity
=====
```

Per-report scores (same report_id = same text, two different scoring prompts):

report_id	Original	Revised	Revised ? Original
1	5.000	5.000	+0.000
2	5.000	4.000	-1.000
3	5.000	4.000	-1.000
4	4.000	5.000	+1.000

--- Summary statistics ---

Original QC rubric (baseline lesson prompt)
Mean = 4.7500 Std dev = 0.5000 (n = 4 reports)

Revised QC rubric (stricter factuality + completeness)
Mean = 4.5000 Std dev = 0.5774 (n = 4 reports)

Mean of (Revised ? Original) across reports: -0.2500

--- Bartlett test (are the two rubrics? score spreads different?) ---

Test statistic: 0.053021
p-value: 0.817887
Meaning: we do **not** reject equal spread at $\alpha = 0.05$ (often a loose check with small n).

--- Paired t-test (do mean scores differ between rubrics on the same reports?) ---

t statistic: 0.522233
degrees of freedom: 3.0
two-sided p-value: 0.637618

Full Pingouin table:
T dof alternative p_val CI95 cohen_d power BF10
0.522233 3 two-sided 0.637618 [-1.27, 1.77] 0.46291 0.102522 0.478

--- What to say in plain language ---

No statistically significant shift between rubrics on **clarity** (paired p = 0.6376). Any mean gap you see could plausibly be random noise at usual $\alpha = 0.05$.

```
=====
MEASURE: succinctness
=====
```

Per-report scores (same report_id = same text, two different scoring prompts):

report_id	Original	Revised	Revised ? Original
1	5.000	5.000	+0.000
2	3.000	5.000	+2.000
3	4.000	2.000	-2.000
4	1.000	4.000	+3.000

```

--- Summary statistics ---

Original QC rubric (baseline lesson prompt)
Mean = 3.2500 Std dev = 1.7078 (n = 4 reports)

Revised QC rubric (stricter factuality + completeness)
Mean = 4.0000 Std dev = 1.4142 (n = 4 reports)

Mean of (Revised ? Original) across reports: +0.7500

--- Bartlett test (are the two rubrics? score spreads different?) ---

Test statistic: 0.090974
p-value: 0.762943
Meaning: we do not reject equal spread at ? = 0.05 (often a loose check with small n).

--- Paired t-test (do mean scores differ between rubrics on the same reports?) ---

t statistic: -0.676481
degrees of freedom: 3.0
two-sided p-value: 0.547222

Full Pingouin table:
T dof alternative p_val CI95 cohen_d power BF10
-0.676481 3 two-sided 0.547222 [-4.28, 2.78] 0.478345 0.106084 0.512

--- What to say in plain language ---

No statistically significant shift between rubrics on succinctness (paired p = 0.5472). Any mean
gap you see could plausibly be random noise at usual ? = 0.05.

=====
MEASURE: relevance
=====

Per-report scores (same report_id = same text, two different scoring prompts):

report_id Original Revised Revised ? Original
1 5.000 5.000 +0.000
2 5.000 5.000 +0.000
3 5.000 5.000 +0.000
4 5.000 5.000 +0.000

--- Summary statistics ---

Original QC rubric (baseline lesson prompt)
Mean = 5.0000 Std dev = 0.0000 (n = 4 reports)

Revised QC rubric (stricter factuality + completeness)
Mean = 5.0000 Std dev = 0.0000 (n = 4 reports)

Mean of (Revised ? Original) across reports: +0.0000

--- Bartlett test (are the two rubrics? score spreads different?) ---

Bartlett is not reliable here (non-finite statistic or nan p-value ? typical when one rubric?s score

```

s have zero spread, e.g. all the same Likert value). Do not use this p-value to draw conclusions.

--- Paired t-test (do mean scores differ between rubrics on the same reports?) ---

Not run: Revised and Original scores are ****identical**** for every report on this measure.
(No variation in paired differences ? t-test is not defined.)

--- What to say in plain language ---

We cannot run a paired t-test here (e.g. every report got the same score under both rubrics). There is no variation in paired differences to test.

=====

MEASURE: overall_six

=====

Per-report scores (same report_id = same text, two different scoring prompts):

report_id	Original	Revised	Revised ? Original
1	4.830	4.500	-0.330
2	4.330	4.000	-0.330
3	4.330	4.000	-0.330
4	3.170	3.830	+0.660

--- Summary statistics ---

Original QC rubric (baseline lesson prompt)
Mean = 4.1650 Std dev = 0.7040 (n = 4 reports)

Revised QC rubric (stricter factuality + completeness)
Mean = 4.0825 Std dev = 0.2896 (n = 4 reports)

Mean of (Revised ? Original) across reports: -0.0825

--- Bartlett test (are the two rubrics? score spreads different?) ---

Test statistic: 1.806853
p-value: 0.178886
Meaning: we do ****not**** reject equal spread at ? = 0.05 (often a loose check with small n).

--- Paired t-test (do mean scores differ between rubrics on the same reports?) ---

t statistic: 0.333333
degrees of freedom: 3.0
two-sided p-value: 0.760820

Full Pingouin table:
T dof alternative p_val CI95 cohen_d power BF10
0.333333 3 two-sided 0.76082 [-0.71, 0.87] 0.15327 0.055744 0.448

--- What to say in plain language ---

No statistically significant shift between rubrics on ****overall_six**** (paired p = 0.7608). Any mean gap you see could plausibly be random noise at usual ? = 0.05.

=====

MEASURE: overall_score

```

=====
Per-report scores (same report_id = same text, two different scoring prompts):

report_id Original Revised Revised ? Original
1 4.830 4.570 -0.260
2 4.330 4.140 -0.190
3 4.330 4.140 -0.190
4 3.170 3.710 +0.540

--- Summary statistics ---

Original QC rubric (baseline lesson prompt)
Mean = 4.1650 Std dev = 0.7040 (n = 4 reports)

Revised QC rubric (stricter factuality + completeness)
Mean = 4.1400 Std dev = 0.3511 (n = 4 reports)

Mean of (Revised ? Original) across reports: -0.0250

--- Bartlett test (are the two rubrics? score spreads different?) ---

Test statistic: 1.155409
p-value: 0.282420
Meaning: we do not reject equal spread at ? = 0.05 (often a loose check with small n).

--- Paired t-test (do mean scores differ between rubrics on the same reports?) ---

t statistic: 0.132237
degrees of freedom: 3.0
two-sided p-value: 0.903168

Full Pingouin table:
T dof alternative p_val CI95 cohen_d power BF10
0.132237 3 two-sided 0.903168 [-0.58, 0.63] 0.044944 0.050494 0.431

--- What to say in plain language ---

No statistically significant shift between rubrics on overall_score (paired p = 0.9032). Any mea
n gap you see could plausibly be random noise at usual ? = 0.05.

=====
EXTRA: completeness (only defined for the revised rubric)
=====
Mean completeness score under the revised rubric: 4.5000

(There is no separate ?completeness? column for the original rubric.)

=====
Done.
=====

```